

JavaScript Architecture & Core Execution

Technical Reference Guide • MAANG Engineering Interview Preparation

Part 1: The Engine & Specifications

Q. What exactly is JavaScript?

A high-level, dynamically-typed, prototype-based, multi-paradigm scripting language. In large-scale architecture, it is defined by its event-driven, non-blocking I/O execution model and first-class functions, allowing it to act as the primary connective logic layer on the web.

Q. What is ECMAScript?

The standardized specification (ECMA-262) that defines the core syntax, native data types, and structural semantics of JavaScript. It acts as the universal blueprint for engines parsing the language.

Q. What is the difference between JavaScript and ECMAScript?

ECMAScript is the theoretical specification; JavaScript is the concrete implementation. JavaScript includes the core ECMAScript standard plus host-specific extensions, such as Web APIs (the DOM, `fetch`) in the browser, or POSIX/OS bindings in Node.js.

Q. What does a JavaScript engine do?

It is the program that interprets and executes JS source code. It parses text into an Abstract Syntax Tree (AST), manages the Call Stack and Memory Heap, and uses a JIT compiler to translate bytecode into highly optimized native machine code.

Q. What is V8?

Google's open-source, high-performance JavaScript and WebAssembly engine, written in C++. It powers Chrome and Node.js. It is renowned for pioneering advanced JIT compilation (Ignition and TurboFan) and efficient garbage collection in the JS ecosystem.

Q. What is the difference between V8 and Node.js?

V8 is strictly the computational engine that executes JavaScript. Node.js is a broader runtime environment that embeds V8 and augments it with a C++ asynchronous I/O library (libuv), enabling JavaScript to access file systems, networks, and OS-level threads.

Q. Can JavaScript run without a browser?

Yes. By embedding a JavaScript engine (like V8 or SpiderMonkey) into a standalone runtime host (like Node.js, Deno, Bun, or custom C++ applications), JavaScript executes independently of the web browser environment.

Q. What is a JavaScript runtime?

The complete architectural environment required to execute JavaScript code. It comprises the JavaScript Engine itself, the Event Loop, task queues (micro and macro), and a suite of host-provided APIs to interact with external hardware or networks.

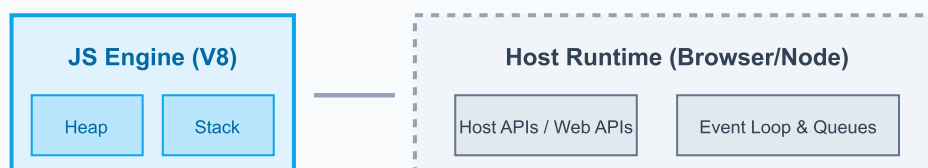


FIG 1. LOGICAL SEPARATION OF THE JS ENGINE AND THE HOST RUNTIME

Part 2: Environments & Concurrency

Q. What is a host environment?

The specific software platform (e.g., Chrome, Node.js, Adobe Acrobat) that embeds the JS engine. It dictates the capabilities of the code by providing global objects (host objects) that bridge JavaScript logic with hardware or OS-level capabilities.

Q. Why does `document` work in a browser but not Node.js?

`document` is the entry point for the DOM (Document Object Model). It is not part of JavaScript itself, but rather a Web API injected into the global scope exclusively by the browser host environment. Node.js operates without an HTML parser and thus lacks a DOM.

Q. Is `setTimeout()` part of JavaScript?

No. It is completely absent from the ECMAScript specification. It is a timer mechanism provided by the host environment (implemented in C++ by browsers as a Web API, and by Node.js via libuv) that pushes callbacks to the task queue.

Q. Is `Promise` part of JavaScript?

Yes. Promises were formally introduced into the core ECMAScript 2015 (ES6) specification to standardize deferred and asynchronous computations directly within the language, replacing disparate third-party libraries.

Q. Is `fetch()` part of ECMAScript?

No. Like `setTimeout`, `fetch()` is defined by the WHATWG Fetch Living Standard. It is a host-provided Web API designed for making asynchronous network requests, returning native ECMAScript Promises to handle the response.

Q. Why is JavaScript called single-threaded?

It executes commands sequentially on a single, primary Call Stack within a single execution realm. This architectural choice prevents complex thread-synchronization issues (like deadlocks or memory corruption) when concurrently manipulating shared resources like the UI/DOM.

Q. Can JavaScript execute on multiple threads?

Yes, indirectly. Modern host environments provide APIs to spin up background threads (Web Workers in browsers, Worker Threads in Node.js). These run in isolated memory realms and communicate with the main thread strictly via serialized message passing.

Q. What is the difference between concurrency and parallelism?

Concurrency is dealing with multiple tasks at once (JavaScript achieves this on a single thread via the Event Loop, interleaving I/O callbacks while waiting). Parallelism is executing multiple tasks simultaneously at the exact same millisecond (requiring multi-core CPUs and Workers).

Feature / Object	Origin Specification	Availability
<code>Array</code> , <code>Map</code> , <code>Promise</code>	ECMAScript (TC39)	Universal (All valid JS engines)
<code>window</code> , <code>document</code> , <code>DOM</code>	WHATWG / W3C	Browsers only
<code>process</code> , <code>fs</code> , <code>http</code>	Node.js API Standard	Node.js (and compatible runtimes) only
<code>setTimeout</code> , <code>console.log</code>	Host Environment Standard	Provided by both Browser & Node hosts

Part 3: JIT Compilation & Typing

Q. Is JavaScript interpreted or compiled?

Neither strictly, it uses a hybrid approach. While historically interpreted, modern engines employ Just-In-Time (JIT) compilation. They initially execute unoptimized bytecode for speed, but actively compile "hot" (frequently run) code paths directly to machine code during execution.

Q. What is JIT compilation?

Just-In-Time compilation translates code into native machine instructions at runtime rather than beforehand. The engine profiles running code, makes speculative assumptions about variable types, and generates highly optimized CPU-specific instructions to match native C++ execution speeds.

Q. What is deoptimization?

A failsafe mechanism in JIT compilers. If the engine's speculative assumptions fail (e.g., a function optimized for integers suddenly receives an object), it discards the optimized machine code ("bails out") and reverts to executing slower, generic bytecode to ensure exact correctness.

Q. Why is JavaScript dynamically typed?

Types are associated with runtime values, not compile-time variables. A single variable can be initialized as a string and later reassigned to a number without explicit type declarations. The engine tracks internal type tags continuously during execution.

Q. Is JavaScript weakly typed or strongly typed?

It is weakly typed. JavaScript routinely performs implicit type coercion, silently converting values between types to force operations to complete (e.g., evaluating `1 + "2"` as `"12"` or `[] == false` as `true`) rather than throwing a type error.

Q. What is the difference between dynamic and static typing?

Dynamic typing (JavaScript, Python) validates and resolves types during application execution, offering rapid iteration but risking runtime type errors. Static typing (TypeScript, Java) strictly enforces type constraints during compile-time, catching mismatches before execution begins.

Q. What are JavaScript's primitive types?

There are seven distinct, immutable primitives: `String`, `Number`, `Boolean`, `Undefined`, `Null`, `Symbol` (introduced in ES6 for unique identifiers), and `BigInt` (introduced in ES2020 for arbitrary-precision integers). They are passed by value, not by reference.

Q. Why does `typeof null` return "object"?

It is a recognized, unfixable historical bug from the first implementation in 1995. Values were stored in 32-bit units, and objects were identified by a `000` type tag. `null` was represented as the machine code NULL pointer (all zeros), so it accidentally matched the object tag.

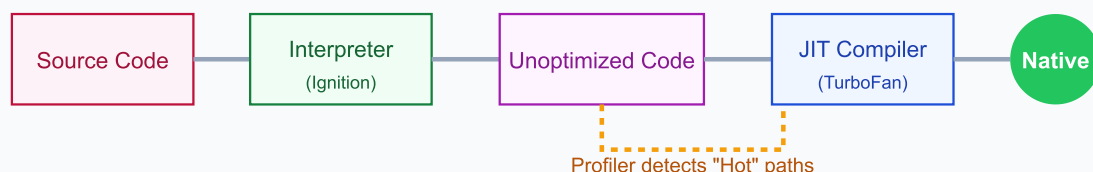


FIG 2. SIMPLIFIED V8 JUST-IN-TIME (JIT) COMPILATION PIPELINE

Part 4: Memory & The Spec Lifecycle

Q. What is garbage collection (GC)?

An automated memory management process built into JS engines. It relies on the "Mark-and-Sweep" algorithm to periodically traverse object graphs from root variables, identifying unreachable data structures and reclaiming their memory heap space to prevent leaks.

Q. How does garbage collection affect performance?

GC operations consume CPU cycles and traditionally require "stop-the-world" pauses, blocking the main thread from executing JS or rendering the UI. Modern engines mitigate this using Generational GC (splitting old/new objects) and Incremental GC (running sweeps in tiny background chunks).

Q. What is TC39?

Technical Committee 39 is a panel of expert delegates (comprising developers from major tech corporations like Google, Apple, Microsoft, and Mozilla) convened by ECMA International. They are responsible for debating, evolving, and standardizing the ECMAScript language specification.

Q. What are ECMAScript proposals?

Formal documents and specifications suggesting new syntax or native APIs for the language (e.g., Optional Chaining, Nullish Coalescing). They undergo rigorous community review and must prove utility, safety, and backwards-compatibility before becoming official syntax.

Q. What are the TC39 proposal stages?

A five-tier maturity pipeline determining a feature's readiness. It starts at Stage 0 (Strawperson) for basic ideas, progressing to Stage 4 (Finished), which indicates full consensus, successful polyfills/implementation, and readiness to be merged into the next annual ES specification.

Q. What is strict mode and why does it exist?

An opt-in, restricted variant of execution triggered by the `"use strict";` directive. It was introduced in ES5 to silently failing errors into explicit throw exceptions (e.g., preventing undeclared global variables) and prohibit confusing, legacy syntax that inhibits JIT engine optimization.

Q. What is sloppy mode?

The colloquial term for JavaScript's default, non-strict execution behavior. It remains the default primarily to guarantee backwards compatibility, ensuring that millions of legacy web scripts written in the 1990s and 2000s do not suddenly crash in modern browsers.

Q. Why are ES modules automatically strict?

When TC39 introduced modern module architecture (`import / export`) in ES6, they mandated that all module bodies execute in strict mode by default. This forced the ecosystem toward safer execution semantics natively, eliminating the need to manually append strict directives to every file.

TC39 Stage	Name	Purpose / Status Requirement
Stage 0	Strawperson	A free-form idea submitted by a TC39 member. Not yet a formal proposal.
Stage 1	Proposal	The committee accepts the problem domain. A champion is assigned to explore solutions.
Stage 2	Draft	Initial spec language is drafted. The API scope and major semantics are defined.
Stage 3	Candidate	The spec is finalized. Needs implementation feedback from browser vendors and users.
Stage 4	Finished	Successfully implemented in two engines. Ready for inclusion in the formal ECMA standard.